

Föreläsning 13

Ändrad
26 Sep
17:59

Någon kanske undrar om pekare verkligen behövs för att funktioner ska kunna ändra en parameter. Någon kanske undrar om man inte kan göra på följande sätt:

```

1  #include <stdio.h>
2
3  void halvera(double y)
4  {
5      y = y / 2;
6  }
7
8  main()
9  {
10     double x = 1;
11
12     do {
13         printf("x = %g\n", x);
14         halvera(x);
15     } while (x > 0.1);
16 }
```

Men nej, det fungerar inte! Av samma anledning som följande kod inte fungerar:

```

x = 1;
y = x;
y = y / 2;    /* x ändras inte! */
```

Vi kan (förstås) inte ändra värde på `x` genom att ändra värde på `y`. De är två olika variabler.

Programexemplet ovan fungerar likadant. Vid anropet av `halvera()` utförs `y = x`. Vad vi sedan gör med `y` påverkar inte `x`.

Minnesregler

Följande två punkter är era minnesregler när ni använder pekare för att ändra parametrar:

1. En `&` före parametern i anropet.
2. En `*` framför parametern inuti funktionen.

Pekare öppnar helt nya dimensioner av felmöjligheter. Glömmer du `&` eller `*`, så blandar programmet ihop minnesinnehåll med minnesadresser. Du läser och skriver på slumpmässiga adresser.

Du kan inte räkna med varningar från kompilatorn. Det bästa som kan hända är att programmet kraschar -- då får du åtminstone veta att nåt är fel.

Variabelparametrar

Andra programspråk -- exempelvis Pascal, C++ och Ada -- har *variabelparametrar*. Om `y` var en variabelparameter skulle programförslaget ovan fungera som förslagställaren tänkt.

Adaprogrammeraren slipper problematiken med pekare och adresser. Istället tar Adakompilatorn hand om allt -- den sätter själv dit `&` och `*` där det behövs.

Funktioner för ordning och reda

Programkoden i nästa exempel har jag delat upp i *fem* funktioner.

Dels för att få mindre att skriva -- `order()` består visserligen bara av två rader, men den drar ner antalet rader i `sort()` till hälften.

Dels för att lättare kunna återanvända det jag gjort -- det är enkelt att klippa ut en funktion ur detta program, och sedan klistra in den i ett annat program. Jag kanske inte ens behöver klippa ut den för att kunna återanvända den ... men mer om detta senare i kursen.

Dels definierar man funktioner därför att det är ett bra sätt att skapa ordning och reda! Av samma anledning som läroboksförfattaren delar in sin bok i kapitel.

Det blir lättare att hitta i bok och program. Och det blir lättare att få översikt. Och lättare att förstå.

Sortering

Följande program sorterar fyra flyttal i storleksordning.

Funktionerna `print()` och `sort()` är bara skolexempel. De är till för att demonstrera olika principer för Er. På riktigt skulle man skriva dem annorlunda.

Funktionerna `order()` och `swap()` är dock meningsfulla, användbara, "riktiga" funktioner, även utanför detta sammanhang

```

1  #include <stdio.h>
2
3  void print(float x0, float x1, float x2, float x3)
4  {
5      printf("%g %g %g %g\n", x0, x1, x2, x3);
6  }
7
8
9  void swap(float *x, float *y)
10 {
11     int temp;
12
13     temp = *x;
14     *x = *y;
15     *y = temp;
16 }
17
18
19 void order(float *x, float *y)
20 {
21     if (*x > *y)
22         swap(x, y); /* &(*x) = x */
23 }
24
25
26 void sort(float *x0, float *x1, float *x2, float *x3)
27 {

```

```

28     order(x0, x1);
29     order(x0, x2);
30     order(x0, x3);
31     order(x1, x2);
32     order(x1, x3);
33     order(x2, x3);
34 }
35
36
37 void main()
38 {
39     float x0 = 400, x1 = 300, x2 = 200, x3 = 100;
40
41     print(x0, x1, x2, x3);
42     sort(&x0, &x1, &x2, &x3);
43     print(x0, x1, x2, x3);
44 }

```

■ `print()` skriver ut fyra flyttal. Finns inget att säga om den.

■ `swap()` byter värden på två variabler. Fungerar precis som `swapchar()` från förra lektionen.

■ `order()` anropar `swap()`, men bara om variablerna är i "fel" ordning.

Lägg märke till att `order()` anropar `swap()` utan adressoperatören `&` framför parametrarna. Det beror på att `x` och `y` redan är adresser -- eftersom de är pekare. Och `swap()` vill ha adresser som argument.

Synbarligen bryter vi mot minnesregel 2 ovan. Men i själva verket har vi använt både regel 1 och 2!

Regel 2 ger `*x`. Ovanpå detta ger regel 1 `&*x` (eller `&(*x)`).

Det är tillåtet att skriva `&*x`. Men tänker man efter inser man att de två operatörerna tar ut varandra.

■ Huvudfunktionen `main()` anropar `print()` och `sort()` med sina flyttalsvariabler. Utskriften antyder att `sort()` lyckas med att sortera sina argument:

```

1  400 300 200 100
2  100 200 300 400

```

För att spåra vad som händer inuti `sort()`, skrev jag en [lite annorlunda version](#) av `sort()` som anropar `print()` mellan varje anrop till `order()`.

Det programmet gav följande utskrift. Jag har kursiverat de förändringar som `order()` infört.

```

1  400 300 200 100
2  300 400 200 100
3  200 400 300 100
4  100 400 300 200
5  100 300 400 200
6  100 200 400 300
7  100 200 300 400

```

Titta på koden till `sort()`:

■ Rad 28--30 garanterar att det minsta värdet hamnar i `*x0`.

- Rad 31--32 garanterar att det närmast minsta värdet hamnar i `*x1`.
- Rad 33 garanterar att det närmast minsta värdet hamnar i `*x2`.
- Därefter finns bara det största värdet kvar -- och det ligger redan i `*x3`.

Allteftersom värden intar sina rätta platser, rör `sort()` dem aldrig mer.

Algoritm, komplexitet

Strategin, metoden, tekniken eller *algoritmen* som `sort()` använder kallas *selection sort* (eng). Algoritmen kan generaliseras till att sortera godtyckligt antal variabler.

- Uppgift: Skissa hur det skulle gå till att sortera tusen variabler enligt algoritmen *selection sort*.
- Fråga: Hur många anrop till `order()` krävs för att sortera n variabler? Svar:

$(n - 1)$ anrop för att hitta det minsta.
 $+$ $(n - 2)$ anrop för att hitta det näst minsta...
 $+$... *ett* anrop för att hitta de två största.
 $= 1 + 2 + \dots + (n - 1) = n(n - 1) / 2$ anrop.

Antalet anrop är en funktion av kvadraten på n . Man säger att *selection sort* har *kvadratisk komplexitet*.

n	<i>selection sort</i>	<i>merge sort</i>
100	5000	714
1000	500000	10466
10000	50000000	137877

Det är ingen imponerande komplexitet. De bästa sorteringsalgoritmerna har komplexiteten $\log_2(n)n$. Jämför i tabellen till höger!

Siffrorna är ungefärliga, trots den synbara precisionen. (Tabellen skapades av [detta C-program](#)).

Nästa lektion

